SUMO Simulation: Lateral Velocity of the Ego, the Adaptive Policy and Identical Runs

Sep 25 – Oct 9

1 Lateral velocity of the ego: old code vs new code

1.1 The issue

- The DIPF potential commands a lateral acceleration $a_y = -\partial U/\partial y$ (up to 3 m/s^2).
- In the **old code** the ego barely moves sideways: lane changes are rare and very slow.
- Cause: the lateral velocity v_y is read back from SUMO every step, and SUMO always returns 0.
- The **new code** keeps v_y as controller state, so the push builds up into real lateral motion.

1.2 Old code: v_y is re-read from SUMO every step

Files: `v2/run1/dipf_controller.py`, `Pure_DIPF_codes.py`.

```
vx = traci.vehicle.getSpeed(EGO_ID)
vy = traci.vehicle.getLateralSpeed(EGO_ID)        # always 0 for the ego

ay = clamp(-dUy, -AY_MAX, AY_MAX)
vy_apply = clamp(vy + ay*dt, -VY_MAX, VY_MAX)     # computed, never stored
y_new = y + vy*dt + 0.5*ay*dt**2                  # vy = 0:  only 0.5*ay*dt**2 is left
traci.vehicle.moveToXY(EGO_ID, ..., x_new, y_new, ...)
```

- The longitudinal speed works: `setSpeed()` stores it in SUMO and `getSpeed()` reads it back.
- There is no equivalent for the lateral speed.

1.3 Why SUMO returns $v_y = 0$

- Sublane model (`-lateral-resolution 0.25`, lane-change model SL2015): lanes are split into 0.25 m sublanes, vehicles can sit at any lateral position and move sideways gradually.
- `getLateralSpeed()` returns the lateral speed **commanded by SUMO's lane-change model**, not the measured motion.
- The ego's lane-change model is disabled (`setLaneChangeMode(ego, 0)`) and the ego is placed with `moveToXY` (a teleport) $\Rightarrow$ no manoeuvre $\Rightarrow$ $v_y = 0$.
- Other vehicles are moved by the lane-change model, so their lateral speed is correct.

Check in SUMO. The ego was moved 0.2 m sideways every step (2 m/s) with `moveToXY` (scenario s05003, SUMO 1.27.1):

step	ego moved sideways	ego `getLateralSpeed`	other vehicles (max)
1	+0.20 m	0.00 m/s	0.10 m/s
4	+0.20 m	0.00 m/s	0.40 m/s
8	+0.20 m	0.00 m/s	0.80 m/s
10	+0.20 m	0.00 m/s	0.50 m/s

1.4 Effect on the motion

Constant lateral push $a_y = 3$ m/s^2, $v_x = 15$ m/s:

time	old code: y	new code: y
0.5 s	0.08 m	0.38 m
1.0 s	0.15 m	1.50 m
1.5 s	0.23 m	3.00 m
3.0 s	0.45 m	7.50 m

- Old: $\Delta y = \frac{1}{2} a_y \Delta t^2 \leq 0.015$ m per step, i.e. about 0.15 m/s of drift (lane width 3.5 m).
- New: v_y builds up to its 3 m/s cap in 1 s; a lane change takes about 1.5 s.

Why the old code still shows some lateral movement

- The $\frac{1}{2} a_y \Delta t^2$ term accumulates under a sustained push (e.g. stuck behind a slow vehicle): 1.5 m after 10 s at 3 m/s^2.
- Fallback in `safe_move_ego`: if `moveToXY` fails, the ego is placed at the nearest lane centre (a jump of up to 1.75 m).
- y is clamped to the safe road band, so the ego is pushed back inside instead of crashing at the edge.

1.5 New code: v_x, v_y kept by the controller

File: `v14/*/dipf_controller.py` (`ego_control_step`).

```
# once, when the ego appears
ego_control_step.vx = traci.vehicle.getSpeed(EGO_ID)
ego_control_step.vy = 0.0

# every step
vx, vy = ego_control_step.vx, ego_control_step.vy
vy_apply = clamp(vy + ay*dt, -VY_MAX, VY_MAX)
vy_apply = clamp(vy_apply, -vx*tan(15 deg), vx*tan(15 deg))   # heading limit
y_new = y + 0.5*(vy + vy_apply)*dt                            # trapezoidal step
traci.vehicle.moveToXY(EGO_ID, ..., x_new, y_new, angle=heading)
ego_control_step.vy = vy_apply                                 # stored for the next step
```

1.6 Summary

	old code	new code
lateral velocity	re-read from SUMO, always 0	controller state, integrated
position	front bumper used as body centre	body centre
heading	chosen by SUMO	from (v_x, v_y), passed to `moveToXY`
v_y vs v_x	no link	$\lvert v_y \rvert \le v_x \tan 15°$
road edge	position clamped to the road	leaving the road = collision
`moveToXY` failure	snap to the nearest lane centre	reported as an error

2 The adaptive policy algorithm (v14)

The DIPF controller itself is unchanged: every acceleration is still the clamped negative gradient of the potential. The adaptive policy only **retunes four of the potential's parameters every** 0.1 s from the traffic around the ego. It never outputs an acceleration, so a bad choice can reshape the field but cannot command an arbitrary manoeuvre.

2.1 Algorithm

Every control step, just before the potential is evaluated:

$$
\begin{aligned}
f_t &= \text{features}(\text{80-value traffic encoding}) \in [0,1]^9 \\
z_t^* &= \text{clip}(W f_t + b,\ -\ln 10,\ +\ln 10) && W \in \mathbb{R}^{4\times 9},\ b \in \mathbb{R}^4 \ \Rightarrow\ 40 \text{ numbers} \\
z_t &= z_{t-1} + \tfrac{\Delta t}{T}\,(z_t^* - z_{t-1}) && \Delta t = 0.1 \text{ s},\ T = 1 \text{ s};\ z_0 = z_0^* \\
p_t &= \text{clamp}(p_0 \cdot e^{z_t},\ \text{limits}) && p = (\lambda_{obs},\ \tau_y,\ \alpha,\ \text{obs_radius_ahead})
\end{aligned}
$$

- p_0 = the trained fixed set from `v14/base` (CMA-ES over all 7 parameters). $W = 0, b = 0$ reproduces it exactly, so any gain comes from adapting.
- e^z: multiplicative and always positive; the clip keeps each parameter within $\times 0.1 \cdots \times 10$ of p_0.
- Smoothing: the exponent moves 10 % of the way to its target each step, so a parameter follows a sudden change within about 1 s instead of jumping.
- The other parameters (s_x, δ_{lane}, y_{margin}, $\tau_x = 1$) stay at p_0.
- Sign constraint: the weights from `ttc_front`, `ttc_side_front`, `drac_front` to λ_{obs} are ≥ 0 (repulsion may only grow with danger).

File: `v14/adaptive/dipf_controller.py` (`adapt_parameters`), `policy.py`.

```
enc = encode((x, y, vx, vy), nearby, road_bounds_y(), EGO_LENGTH, EGO_WIDTH)
f = pol.features(enc)                                    # 9 features in [0, 1]
z_target = clip(W @ f + b, -ln10, ln10)
z = z_target if z is None else z + min(1, dt/T) * (z_target - z)   # smoothing
for k, name in enumerate(params):                        # lambda_obs, tau_y, alpha, radius
    value[name] = clamp(base[name] * exp(z[k]), *limits[name])
# ... then the unchanged DIPF step: U, grad U, a = clamp(-grad U), moveToXY
```

2.2 Inputs: nine features

All in $[0, 1]$, 0 = nothing there / far away, and unchanged when left and right are swapped (like the controller).

feature	definition	large when
`ttc_front`	$1 - e^{-1/\mathrm{TTC}}$, vehicle in the ego's path ahead	closing in fast ahead
`ttc_side_front`	same, worst of front-left / front-right	closing on a diagonal vehicle
`ttc_rear_side`	same, worst of rear, rear-left/right, alongside	closing from behind/side
`drac_front`	$1 - e^{-\mathrm{DRAC}/3}$, braking for the car ahead	hard braking needed
`gap_front`	$e^{-\mathrm{gap}/20\,\mathrm{m}}$	vehicle close ahead
`gap_side_front`	same, closest of front-left / front-right	vehicle close diagonally ahead
`tight_lateral`	$e^{-\mathrm{room}/1\,\mathrm{m}}$ (road edge or side gap)	no room to swerve
`speed`	$v_x/34$ m/s	driving fast
`density`	$1 - e^{-n/10}$, n = vehicles within ±50 m	crowded

2.3 Training

- No target value of p_t exists, so the 40 numbers are trained directly on closed-loop outcome: J = success rate + $0.1 \cdot$ mean progress, takeover ($x \approx 100$ m) to $x = 400$ m.
- CMA-ES from $W = 0, b = 0$: 30 generations × 15 candidates, each scored on the same 100 train scenarios × 2 futures (same budget as the fixed base: 450 candidates).
- Every 4 generations the search mean is scored on validation (100 × 3 runs); the best one is kept (generation 12, validation success 0.730 vs 0.647 for the fixed set).

2.4 What it learned

- λ_{obs} (repulsion): rises sharply with a vehicle closing from behind/side (`ttc_rear_side` +2.0) or diagonally ahead (+1.85), and with tight lateral room (+1.3); falls behind a close but non-closing car (`gap_front` −1.7), so the ego follows calmly.
- τ_y (lateral reach): grows with speed (+1.5), shrinks when closing in ahead (−1.3).
- α: pinned at its 0.01 floor in every step, so it does not really adapt.
- Look-ahead radius: about ×1.8 of the fixed set (≈ 174 m vs 97 m), shorter in dense traffic.

2.5 Results

Test: 300 fresh scenarios × 3 futures = 900 paired runs (identical traffic for every controller, Section 3).

controller	success	progress
v12 library set 2 (v13 base)	0.588	0.750
trained fixed set (`v14/base`)	0.646	0.780
adaptive offsets only ($W = 0$)	0.621	0.751
adaptive policy (`v14/adaptive`)	**0.714**	**0.812**

- Adaptive vs trained fixed set: $+6.9 \pm 2.2$ points (only adaptive succeeded 182, only fixed 120; McNemar $p = 0.0004$).
- The gain comes from adapting: the offsets alone are no better than the fixed set (-2.4, $p = 0.21$).

3 Identical SUMO scenarios and futures (v14)

All v14 comparisons (base vs adaptive, the numerator variants) rely on every candidate meeting **exactly the same traffic**. Three pieces make this possible: saved scenarios, futures, and repeatable SUMO runs.

3.1 Scenarios: saved starting states

- Created once by `step1_make_scenarios.py`: traffic runs for 30 s, the car nearest $x = 100$ m becomes the ego, and the full SUMO state is saved at that moment.
- Per scenario `scenarios/sNNNNN/`: `state.xml` (all vehicles, routes, RNG state), `ego.json` (the controller-held ego state v_x, v_y, heading), `meta.json` (seed, road length, start x).
- Saved **before** the controller acts, so a scenario does not depend on the DIPF parameters or the potential variant.
- All v14 sub-folders share one scenario folder, `v14/base/scenarios` (their `config.yaml` points `scenarios_dir` to it).
- Seed splits: train 0–99, validation 400–499, test 5000–5299 (never used for training or selection).
- Guard: `meta.json` records the road length; every step refuses scenarios made on another road.

3.2 Futures: several random continuations of one scenario

- A scenario fixes the **start**; the traffic after it is random (driver imperfection, speed deviation, lane-change decisions).
- Each scenario is reloaded with SUMO seeds $1, \ldots, n$ (`-seed`): **future** k = scenario + seed k. Same start, different random continuation.
- Futures per scenario: train 2, validation 3, test 3 $\Rightarrow$ one result is a success *rate*, not a single lucky or unlucky run.
- **Common random numbers**: every candidate (parameter set or policy) is run on the same scenarios $\times$ futures, so differences come from the controller, not from the traffic. Results are compared run by run (paired McNemar test).

```
cmd = [sumo, "-n", NET_FILE.name, "--load-state", "scenarios/sNNNNN/state.xml",
       "--step-length", "0.1", "--lateral-resolution", "0.25", ...]
cmd += ["--seed", str(future)]                      # future 1, 2, 3, ...
```

3.3 Repeatable runs: same scenario + same future $\Rightarrow$ same result

Problem 1: address-space randomisation (ASLR). SUMO orders some internal work by memory address. Linux randomises memory addresses on every launch, so the same run (state, seed, commands) drew its random numbers in a different order each time: after one step two vehicles 300 m apart had swapped their speed perturbations, and an identical run could end in success once and in a collision the next time.

10 repeats of the same run	default (ASLR on)	v14 (ASLR off)
s00007, future 2	2 different outcomes	1
s00003, future 1	4 (2 successes, 2 collisions)	1
s00012, future 2	2 different outcomes	1

Fix: every SUMO process is started with address randomisation disabled (equivalent of `setarch -R`):

```
def _kill_with_parent():               # preexec_fn: runs just before SUMO starts
    libc = ctypes.CDLL("libc.so.6", use_errno=True)
    libc.prctl(1, signal.SIGKILL)                                  # SUMO dies with its parent
    ADDR_NO_RANDOMIZE = 0x0040000                                  # no ASLR:
    libc.personality(libc.personality(0xFFFFFFFF) | ADDR_NO_RANDOMIZE)
```

Problem 2: the result depended on the folder. SUMO's memory layout also depends on the length of its command line and environment. The same parameters run from `v14/base` and `v14/adaptive` gave different outcomes (network path `.../base/...` vs `.../adaptive/...`, and the `PWD` variable). Fix: identical command line and environment from every folder:

```
env = {k: v for k, v in os.environ.items() if k not in ("PWD", "OLDPWD", "_")}
# relative "-n" path, run in the network's folder, fixed environment
proc = subprocess.Popen(cmd + ["--remote-port", str(port)], preexec_fn=_kill_with_parent,
                        cwd=str(NET_FILE.parent), env=env)
```

Checks.

- Repeating a run gives the identical outcome and progress (30 of 30 repeats).
- The same parameters give identical results from `v14/base` and `v14/adaptive`.
- The adaptive policy with all weights 0 reproduces the trained fixed set run by run, and `adaptive/step3` reproduces `base/step3` exactly (success 0.646, progress 0.780).
- All runs must use the same SUMO binary: the scenarios were made with SUMO 1.27.1 (venv); SUMO 1.18 (`/usr/share/sumo`) cannot load them. `sumolib.checkBinary` follows `SUMO_HOME`.